AI Document Classifier

Final Project Report

MPOnline Internship Program

Artificial Intelligence & Machine Learning

Submitted by: Akarshika Singhal

Date: 30th June, 2026

Framework: TensorFlow / Keras | Backend: FastAPI | Frontend: React

97.33%	Best Model Accuracy
1,500	Augmented Images
3	Document Classes
3	Models Compared

Table of Contents

1. Project Overview

This project was developed as part of the MPOnline Internship Program in Artificial Intelligence and Machine Learning. The goal was to design, develop, train, and demonstrate an AI-powered document classification system capable of identifying three types of documents: Aadhaar Cards, PAN Cards, and Marksheets from uploaded images.

The system uses deep learning-based transfer learning with pre-trained convolutional neural networks (CNNs) to classify documents with high accuracy. The project covers the complete software development lifecycle — from data collection and augmentation to model training, backend API development, and frontend UI creation.

Technology Stack

Component	Technology	Purpose
AI / ML Framework	TensorFlow 2.6 / Keras	Model training & inference
Model Architecture	MobileNetV2, ResNet50, VGG16	Transfer learning
Backend Framework	FastAPI (Python)	REST API development
API Documentation	Swagger UI (auto-generated)	Interactive API docs
Frontend	React (HTML/JS)	User interface
Data Processing	OpenCV, NumPy, Pillow	Image preprocessing
Development Environment	Anaconda + Jupyter Notebook	Experimentation
Version Control	GitHub	Code repository

2. Objectives

Document Upload & Classification

Allow users to upload document images and classify them into Aadhaar Card, PAN Card, or Marksheet categories with high accuracy.

Confidence Score Generation

Generate confidence scores for each classification, providing transparency into model predictions and all class probabilities.

FastAPI Backend Development

Build a robust REST API with Upload, Predict, Health Check, and Model Info endpoints with Swagger documentation.

Model Comparison

Train and compare multiple deep learning models (MobileNetV2, ResNet50, VGG16) to justify the final model selection.

Modern User Interface

Develop a clean, responsive React-based frontend with drag-and-drop upload, real-time results, model switching, and prediction history.

Model Switching

Implement a model switcher API allowing dynamic switching between trained models without restarting the server.

3. Dataset Details

The dataset was collected manually and consists of real-world document images across three categories. Images were sourced from various angles, lighting conditions, and qualities to improve model robustness.

Document Class	Original Images	After Augmentation	Percentage
Aadhaar Card	59	500	33.3%
PAN Card	68	500	33.3%
Marksheet	73	500	33.3%
Total	**200**	**1,500**	**100%**

Dataset Characteristics

- Images collected from real documents with varying quality, angles, and lighting
- All images resized to 224 x 224 pixels for model compatibility
- Dataset split: 80% training (1,200 images) and 20% validation (300 images)
- Balanced classes ensured fair training across all document types
- Supported formats: JPG, JPEG, PNG, BMP, WEBP

4. Data Augmentation

Since the original dataset contained only 200 images, data augmentation was applied to expand the dataset to 1,500 images (500 per class). Augmentation was implemented using OpenCV and applied a random combination of 2 to 4 transformations per image to simulate real-world variation.

Augmentation Technique	Parameters	Purpose
Rotation	±20 degrees	Simulate tilted scans
Horizontal Flip	Random	Mirror variations
Brightness Adjustment	Factor: 0.6 – 1.4	Poor lighting conditions
Contrast (CLAHE)	Factor: 0.7 – 1.3	Over/under-exposed scans
Gaussian Noise	Std: 5 – 25	Camera grain simulation
Zoom	0.85x – 1.15x	Different capture distances
Shear	±0.15	Perspective distortion
Gaussian Blur	Kernel: 1, 3, or 5	Out-of-focus simulation
Hue/Saturation Shift	Hue ±10, Sat 0.7–1.3	Color temperature variation
Perspective Warp	Distortion: 5%	Angled document photos

5. Model Architecture & Training

Three pre-trained CNN models were used with transfer learning. The base models were loaded with ImageNet weights and their layers frozen. Custom classification heads were added on top of each base model for the 3-class document classification task.

Training Configuration

Parameter	Value
Image Size	224 x 224 pixels
Batch Size	16 (optimized for CPU)
Epochs	20 (with EarlyStopping)
Learning Rate	0.0001 (Adam optimizer)
Loss Function	Categorical Crossentropy
Validation Split	20% (300 images)
Training Split	80% (1,200 images)
Random Seed	42 (reproducibility)

Training Callbacks

EarlyStopping: Monitors validation loss and stops training if no improvement for 5 consecutive epochs, restoring best weights.

ModelCheckpoint: Saves the best model automatically based on validation accuracy during training.

ReduceLROnPlateau: Reduces learning rate by 50% when validation loss plateaus for 3 epochs, with minimum LR of 1e-7.

MobileNetV2

Base Model	152 frozen layers, GlobalAveragePooling2D
Classification Head	Dense(128, ReLU) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.2) → Dense(3, Softm
Total Parameters	2,430,403

ResNet50

Base Model	175 frozen layers, GlobalAveragePooling2D
Classification Head	Dense(256, ReLU) → Dropout(0.4) → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Soft
Total Parameters	24,145,539

VGG16

Base Model	19 frozen layers, Flatten
Classification Head	Dense(256, ReLU) → Dropout(0.5) → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Soft
Total Parameters	21,170,755

6. Model Comparison & Results

All three models were trained on the same augmented dataset with identical training configuration. The results were compared to select the best model for the production backend.

Metric	MobileNetV2	ResNet50	VGG16
Val Accuracy	96.0%	72.33%	**100.0%**
Val Loss	0.0972	0.6165	0.0086
Epochs Trained	20	20	16
Total Parameters	2,430,403	24,145,539	21,170,755
Training Time	~25 mins	~45 mins	~55 mins
Model Size	Lightweight	Heavy	Heavy
Recommended For	Production	GPU training	High accuracy

Model Selection Justification

Although VGG16 achieved 100% validation accuracy, MobileNetV2 was selected for the production FastAPI backend for the following reasons:

- MobileNetV2 has only 2.4M parameters compared to VGG16's 21M — making it 8x lighter
- MobileNetV2 achieved 96% accuracy which is excellent for a 3-class problem
- Faster inference time on CPU — critical for real-time document classification
- Lower memory footprint — suitable for deployment on limited hardware
- VGG16's 100% accuracy on the validation set may indicate overfitting on small datasets

7. FastAPI Backend

The backend was developed using FastAPI (Python), providing a high-performance REST API for document classification. Swagger UI documentation is auto-generated and accessible at http://127.0.0.1:8001/docs.

Method	Endpoint	Description
GET	/	Root — API welcome message & available endpoints
GET	/health	Health check — server status, model loaded, framework
GET	/model-info	Model details — accuracy, parameters, classes, epochs
POST	/switch-model/{name}	Switch active model — MobileNetV2 / ResNet50 / VGG16
POST	/upload	Upload document — returns file info & confirmation
POST	/predict	Classify document — returns class, confidence, probabilities
POST	/compare	Compare all models — runs prediction on all 3 models

Key Features

- Model caching — once loaded, models are cached in memory for fast switching
- Confidence level labeling — Very High (90%+), High (75%+), Medium (60%+), Low (<60%)
- CORS enabled — allows frontend to communicate with backend across origins
- Input validation — rejects non-image files with descriptive error messages
- Swagger UI — interactive API documentation auto-generated at /docs

8. React Frontend

A modern, responsive frontend was developed using React (via CDN — no build tools required). The interface provides an intuitive document upload experience with real-time classification results.

Feature	Description
Stats Dashboard	Live display of active model, accuracy, prediction count, and classes
Model Switcher	One-click switching between MobileNetV2, ResNet50, and VGG16
Drag & Drop Upload	Upload documents by dragging or clicking with image preview
Classification Result	Predicted class with icon, confidence bar, confidence level badge
Probability Breakdown	Visual bar chart showing probability for all 3 classes
Prediction History	Last 10 predictions with class, confidence, model, and timestamp
Server Status Indicator	Live green/red dot showing if FastAPI server is reachable
Toast Notifications	Success/error messages for user actions
Responsive Design	Works on desktop and mobile screens

9. System Architecture

The system follows a three-tier architecture with a React frontend, FastAPI backend, and TensorFlow model layer. The components communicate via REST APIs over HTTP.

Layer	Component	Technology	Role
Presentation	React Frontend	HTML / React / JS	User interface & interaction
Application	FastAPI Backend	Python / FastAPI	REST API & business logic
ML Model	CNN Classifier	TensorFlow / Keras	Document classification
Data	Augmented Dataset	OpenCV / NumPy	Training data
Storage	Saved Models	.h5 files	Persisted trained models

Data Flow

Step 1: User uploads a document image via the React frontend

Step 2: Frontend sends a POST request to FastAPI /predict endpoint

Step 3: FastAPI reads the image, resizes to 224x224, and normalizes pixel values

Step 4: Preprocessed image is passed to the active TensorFlow model

Step 5: Model outputs probability scores for all 3 document classes

Step 6: FastAPI returns predicted class, confidence score, and all probabilities

Step 7: Frontend displays results with visual confidence bar and probability breakdown

10. Conclusion & Future Scope

Conclusion

This project successfully demonstrates an end-to-end AI-powered document classification system. The system achieves excellent classification accuracy using transfer learning with MobileNetV2 (96%) and VGG16 (100%) on a dataset of only 200 original images expanded through data augmentation. The FastAPI backend provides a robust, well-documented REST API and the React frontend delivers an intuitive user experience with model switching capabilities.

The project satisfies all requirements of the MPOnline AIML Internship — document classification, confidence score generation, FastAPI backend with Swagger documentation, model comparison, and a modern user-centric frontend.

Key Achievements

- Trained 3 deep learning models and compared their performance systematically
- Achieved 96% accuracy with MobileNetV2 and 100% with VGG16
- Built a production-ready FastAPI backend with 7 REST endpoints
- Implemented dynamic model switching without server restart
- Developed a clean, responsive React frontend with drag-and-drop upload
- Generated 1,500 augmented images from 200 originals using 10 augmentation techniques
- Deployed Swagger UI for interactive API documentation and testing

Future Scope

- Add JWT authentication for secure API access
- Deploy the system to a cloud platform (AWS / GCP / Azure)
- Expand document classes to include Driving Licence, Passport, and Voter ID
- Implement OCR (Optical Character Recognition) to extract text from classified documents
- Add a database to store prediction history and user activity logs
- Train on a larger dataset for even higher accuracy and better generalization
- Build a mobile application using React Native for on-device document scanning
- Implement batch prediction for processing multiple documents simultaneously

Project Repository: GitHub | **Backend:** FastAPI + Swagger UI | **Framework:** TensorFlow / Keras